Task 1: Anomaly detection in an IoT setting (spotlight: Stream processing)

IU International University of Applied Science

Written report for oral exam

Degree Program: Data Science

Project: From Model to Production (DLBDSMTP01)

Tutor: Frank Passing

Date: 29/10/2026

Author: Devendra Shrestha

Matriculation number: 9216496

Table of Contents

Table of Contents

Table of Contents ..... *I*

List of Figures ..... *III*

1. Introduction *1*

2. Company Background and Business *2*

3. Business Problem *3*

4. Project objectives *4*

5. Project Planning and System Design *5*

6. Data generation and preparation *6*

7. Machine Learning Model Development *7*

8. Model Testing and Performance Evaluation *7*

9. API Development and system Integration *9*

10. Stream Processing Simulation 10

11. Business Benefits of the Proposed Solution 11

12. Challenges and Limitations 11

13. Monitoring and Future Development 12

14. Personal Learning and Reflection 13

15. Conclusion 14

16. References 16

List of Figures

Figure 1:Architecture of the IoT anomaly detection prototype..... 5

1. Introduction

I developed this project to investigate how machine learning and IoT technology can help a manufacturing company identify unusual production conditions. The project focuses on wind turbine component manufacturing, where monitoring temperature, humidity and sound measurements can support production quality and operational efficiency.

When producing items in a company there will always be anomalies that occur during the process. If we can find these anomalies we can allow our workers to see what is going on in case it becomes a bigger problem.

For this project, I created an anomaly detector using python and machine learning. I had no access to data from an actual factory, so I created my own synthetic dataset. My dataset contained 3,000 manufactured items that each had temperature, humidity and sound measurements.

I then created a machine learning model using isolation forest algorithm to detect unusual patterns in sensor measurements. After training and testing my model I created a REST API with FastAPI to implement my model into.

A python script was created that would continuously send sensors measurement values at fixed interval. The API would take in these measurements and give me an anomaly prediction and score.

My model was able to get 80.8% Precision, 95.5% Recall and an F1-score of 87.5% on my synthetic data. With this project I have shown how a company can implement machine learning to

detect anomalies. Allowing companies to better monitor their machines and give them more information when making decisions.

2. Company Background and Business

Manufacturing companies want to implement technology into their business to help them be more efficient and create a better product.. We can obtain data from our machines by using technology such as IoT, Artificial intelligence/Machine learning that can allow us to obtain and use data.

In wind turbine component manufacturing, maintaining consistent production conditions is important because the components must meet technical and quality requirements.

While producing these parts you can collect data from various sensors. These sensors can measure temperature of your working area, humidity and even sound. All this data can be collected while the parts are being manufactured.

You can then use this data to see if your production is running normal or if there are anomalies occurring.

I chose to use the manufacturing of wind turbine parts as my company. There are many ways you can collect data from a manufacturing facility. I wanted to create a solution that would help a company detect anomalies with their sensor data.

This is not a real-world company that I am trying to implement my solution on. I created a simulated factory to show how the proposed solution could work.

3. Business Problem

What I found to be the main business problem was being able to monitor many sensor data and determine the unusual production conditions quickly.

In a manufacturing company, production equipment may generate measurements throughout the working day. If employees need to examine all this information manually, the process can become time-consuming and inefficient.

Usually there will be set values for monitoring. The company would create a high value for temperature and if it went over that it would send out an alert.

While this could help find some issues. It would not find anomalies that consist of multiple different measurement outputs.

Maybe temperature and humidity as well as a sound output may individually appear within normal ranges but together they may not be typical for what your used to seeing in a manufacturing setting.

This is where I wanted to see how machine learning could help to solve this problem.

By creating a model that will detect anomalies from sensor outputs. Find those outputs that are different from what we consider normal.

From a business perspective, this could allow a manufacturing company to have better monitoring and help with finding where to start looking for potential problems.

4. Project objectives

As stated above I wanted to build a machine learning anomaly detection prototype that would detect anomalies in a IoT Manufacturing setting.

Here are some goals that I wanted to accomplish:

First, I wanted to create a dataset of manufacturing sensors measurements. Because I did not have access to measurements from an actual company I decided to make fake measurements using python.

Second, I wanted to build a machine learning algorithm that can predict anomalies from temperature, humidity and sound data.

Third, I wanted to develop an API that will receive sensor measurements and return predictions automatically.

Fourth, I wanted to Simulate a continuous flow of events happening at a factory and send them to my program.

Finally, I wanted to evaluate the performance of the model and identify improvements required before the system could be considered for real industrial uses.

I hope, I practically connect the manufacturing problem with a technical solution using machine learning and stream-processing concepts.

5. Project Planning and System Design

When creating this project, I wanted to create the overall system structure and identified the components required to complete the project.

There are 3 sections of my system. Data generation, machine learning and API-based prediction.

The first part generates simulated sensor measurements. The second part trains and stores the anomaly detection model. The third part receives new measurements and produces predictions.

By splitting up the training of my model and making predictions I can train my model once and use the saved model to make predictions when my API receives a new sensor measurement.

This system allows for me to plug in real IoT sensor data in the future.

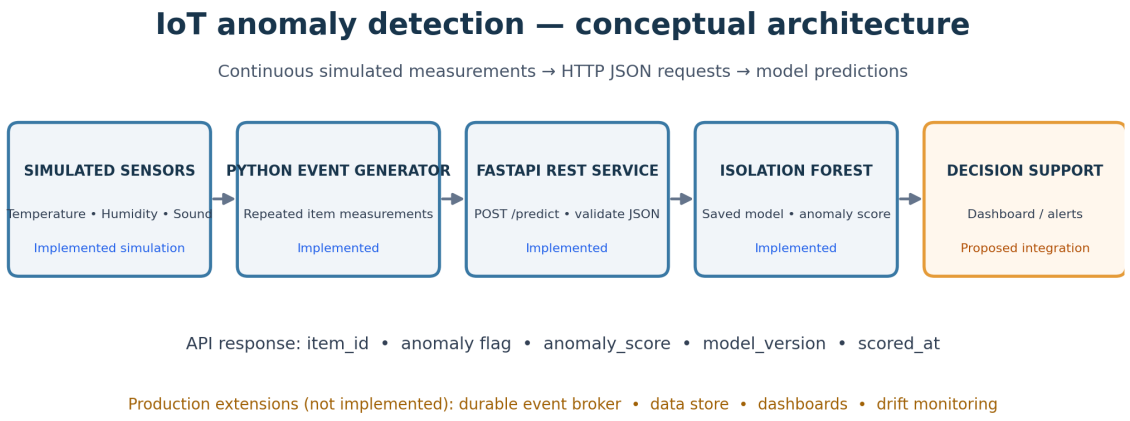


Figure 1:Architecture of the IoT anomaly detection prototype

I selected this architecture because it is relatively simple, modular and suitable for demonstrating the complete process from sensor measurement to anomaly prediction.

6. Data generation and preparation

One of the first tasks I completed was preparing the dataset for machine learning.

Because I did not have access to actual manufacturing equipment or industrial sensors, I generated synthetic data using Python.

I created 3,000 independent manufacturing item records. Each record contained three measurements: temperature, humidity and sound.

I selected these features because they can represent important environmental and operational conditions in a manufacturing process.

I used the following baseline values:

Sensor measurement	Baseline value
Temperature	65°C
Humidity	48%
Sound level	73 dB

I generated measurements around these baseline values using normal distributions.

To represent abnormal production conditions, I introduced larger changes in one measurement for approximately 8% of the items.

I also restricted humidity measurements to valid percentage values.

To ensure that the dataset could be reproduced, I used a fixed random seed of 42.

After generating the data, I divided the dataset into two groups. I used 2,400 records for training and 600 records for testing.

I kept the synthetic anomaly labels separate from the model training process because Isolation Forest is an unsupervised learning algorithm. I used the labels only to evaluate the model's predictions.

This approach allowed me to test whether the model could identify unusual observations without being directly trained on anomaly labels.

7. Machine Learning Model Development

Once my data was cleaned I created a machine learning model.

I used the Isolation Forest algorithm to find outliers. The isolation forest will detect points that may be far from other points.

This model uses random trees to isolate observations that can be isolated with less amount of splits.

I thought this would be a good fit for this problem since we were trying to find anomalies in sensors not what kind of anomaly it could cause when making our product.

I configured the model with 150 trees and a contamination parameter of 0.08.

The contamination parameter reflected the approximate proportion of abnormal observations introduced into my synthetic dataset.

I trained the model using temperature, humidity and sound measurements in a consistent feature order.

After training, I saved the model using **joblib** so that it could be loaded by the API without repeating the training process.

I implemented the training procedure in the **train.py** script.

This separation between model training and API prediction was important because it allowed me to manage the machine learning component independently from the application that receives sensor measurements.

8. Model Testing and Performance Evaluation

After I trained my model I wanted to see how well it performed. I tested my model against the 600 test records.

By finding the precision recall and f1 score and confusion matrix I can determine how well my model predicted the synthetic anomaly's.

Here are the results from my testing.

80.8% ⟹Precision

95.5% ⟹ Recall

87.5% ⟹ F1-score

As you can see from the confusion matrix above we were able to predict 546 of the normal items and find 42 of our anomalous items.

However, it also incorrectly flagged 10 normal items as anomalies and failed to detect 2 anomalous items.

I believe these results are good for a synthetic demonstration.

The high recall indicated that the model detected most of the deliberately introduced anomalies. However, the false positives showed that some normal production measurements could still trigger unnecessary alerts.

In a real-world scenario if we got false positives we would waste time inspecting items. False Negatives would cause us to miss abnormal situations in our production line.

These results will not translate to an actual factory we would need to use measurements from sensors and verify that they had quality issues with their product.

9. API Development and system Integration

Once my machine learning model was created, I created a REST API using FastAPI.

This would allow another application to send sensor measurement to the model and receive predictions.

I created an endpoint for prediction which is called POST /predict.

This endpoint receives an item identifier together with temperature, humidity and sound measurements.

When the API receives a request, it validates the information and sends the measurements to the trained Isolation Forest model.

The model returns a prediction, and the API responds with an anomaly flag, anomaly score, model version and timestamp.

I also implemented a health endpoint called GET /health to check whether the model had been loaded successfully.

I used Pydantic validation to ensure that incoming measurements followed the expected format and configured value ranges.

If the request contains invalid information, the API returns an HTTP 422 validation error.

I considered this validation important because manufacturing monitoring systems need consistent and reliable data.

The API also provides a clear integration point for future applications, such as factory dashboards or automated alert systems.

10. Stream Processing Simulation

Below you see how I was able to stream-process data with my project.

I created a python script that will output item level manufacturing measurements to the prediction endpoint on FastAPI prediction endpoint.

I configured the simulator to send a new request every 0.5 seconds by default.

For the demonstration, I used the following command:

python **simulate_stream.py --items 30 --interval 0.5**

This command sends 30 simulated manufacturing events to the API.

For each event, the API validates the measurements, runs the anomaly detection model and returns a prediction.

This allowed me to demonstrate how the system responds to a sequence of incoming sensor events.

However, I understand that the simulator does not represent a complete industrial stream-processing platform.

The measurements are already aggregated for each manufactured item, and the application does not perform event-time windowing or use a message broker.

In a real manufacturing company, sensors could generate many readings per second. A production system would need to collect, synchronize and aggregate these measurements before applying the anomaly detection model.

I would consider using MQTT or Apache Kafka for this purpose in a future version.

11. Business Benefits of the Proposed Solution

Here are some potential benefits I think the company could see from using an anomaly detection system.

The first benefit that they could monitor their production better. They would be able to look at all there sensor measurements and identity unusual conditions.

The second benefit they could start looking into their production line if the system detects an anomaly. The worker could investigate the specific product being produced.

The third benefit they could make better decisions with anomaly detection. Anomaly detection results could provide additional information to quality control teams and production managers.

Another benefit if they kept all their anomalies in a data base they could study past anomalies and improve their production line.

The system could also support the development of more advanced industry 4.0 applications.

I know that these are all potential business benefits that my prototype could achieve. I haven't shown that we can decrease defect rates or time down or cost of production.

Those benefits would need to be measured through testing in a real manufacturing environment.

12. Challenges and Limitations

Here are some issues that I found while doing this project that would have to be improved if we wanted to implement this into a manufacturing company.

The most important limitation was the use of synthetic data. Although it allows me to develop and evaluate the prototype, it did not full represent the complexity of real manufacturing conditions.

There were not many streaming features implemented in my simulator. It only makes multiple HTTP requests to simulate the stream. It could be better if it was able to send raw measurements from sensors.

There should be standardized measurements and features being passed to the API. If measurements are coming in with different units or the wrong feature ordering the results may not be accurate.

If there are network issues, events sent twice or measurements being taken too late. These are all problems that could occur when implementing this in the real world.

Furthermore, my prototype does not include authentication, persistent storage or a production monitoring dashboard.

What if the normal values change? There would have to be a way to train your model on newer data.

I learned many things when trying to make a working prototype and what it takes to implement this into a factory.

13.Monitoring and Future Development

If I had more time to work on this project for a manufacturing business I would focus on improving reliability, scalability and practical usability.

My first improvement I would incorporate actual with real IoT devices to measure manufacturing processes.

Later, I would then introduce a message broker such as MQTT or Apache Kafka to handle continuous sensor ingestion.

Using event-time I would be able to cluster together all the measurements taken for one product being created.

I would also implement a database to save sensor measurements and predicted outputs.

The company would then be able to see what may have happened before if any anomalies occur.

I would also create some sort of dashboard for those working at the factory. There could be graphs and values shown of sensors. As well as anomaly predictions and previous history.

For system monitoring, I would track API error rates, anomaly rates, response times and changes in sensor data distributions.

Finally, I will implement security measures and allowing the model to be updated with new versions.

All these changes would allow me to take my project a step further towards monitoring factories.

14.Personal Learning and Reflection

From this project I learned about implementing machine learning into a monitoring system for manufacturing.

I learn that building a machine learning model is just a part of the complete solution.

There were many other factors I had to think about when creating this project.

Such as creating the data, application, API communication, validating and testing your model.

Creating my own synthetic dataset allowed me to learn how to create data for machine learning. And use artificial anomalies to evaluate our model.

Training the isolation forest allowed me to learn about how you can use an unsupervised

algorithm to find anomalies. Without knowing what an anomaly is while training.

Building my FASTAPI application allowed me to understand how a trained model can be made available to another software component.

Learning about stream simulation allowed me to understand how incoming events can be processed repeatedly though an API.

Learning about evaluating your model. My model worked well for my generated data but when it comes to real world manufacturing data it may yield different results.

Another thing to note about predicting anomalies. Is that just because it predicted an anomaly doesn't mean it is a bad manufactured product. It just means that it’s out of the ordinary based off what data you trained it on.

overall, this project allowed me to learn more about Machine learning, IoT concepts, software integration and practical challenges of building data-driven decision solution for manufacturing company.

15.Conclusion

For this project I built a prototype anomaly detection system for a simulated IoT manufacturing scenario.

My goal for this project was to explore machine learning to see if it can detect anomalies on a manufacturers company. By collecting temperature, humidity and sound data from manufactured products.

I generated a synthetic dataset containing 3,000 manufacturing items and trained an Isolation Forest to identity unusual patterns.

After testing on a test dataset, I achieved a precision score of 80.8% recall score of 95.5% and F1-score of 87.5%.

I then created a FastAPI application to receive sensor measurements and return anomaly detection results.

I wrote a python script to simulate data flowing through this pipeline by continuously sending products through the API.

This allowed me to create a use case where i can gather data from products, run machine learning on it and create an API that can be accessed by a stream processor.

In a real-world scenario this could help a manufacturer by giving them the ability to see when anomalies may be occurring in their production line. Allowing them to troubleshoot why products are behaving oddly.

Of course, more work would have to be done to make this system viable in a production environment. Connecting actual sensors to a streaming platform that can ensure throughput and have measures in place to keep data secure.

16.References

FastAPI. (n.d.). *FastAPI documentation*. https://fastapi.tiangolo.com/

NumPy Developers. (n.d.). *NumPy documentation.* https://numpy.org/doc/

Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3/

Scikit-learn Developers. (n.d.). *IsolationForest.* Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

Project source code can be found on my own GitHub repository: https://github.com/Devgita/Anomaly-detection-in-an-IoT-setting-spotlight-Stream-processing-